

NeuralNetwork

Generated by Doxygen 1.15.0

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 ActivationFunction Struct Reference	5
3.1.1 Detailed Description	5
3.2 ColumnData Struct Reference	5
3.2.1 Detailed Description	6
3.3 Configuration Struct Reference	6
3.3.1 Detailed Description	6
3.4 DataPreprocessor Class Reference	6
3.4.1 Detailed Description	7
3.4.2 Member Function Documentation	7
3.4.2.1 fit()	7
3.4.2.2 interpret_extracted_column()	8
3.4.2.3 transform_and_extract()	8
3.5 LossFunction Struct Reference	9
3.5.1 Detailed Description	9
3.6 NeuralNetwork Class Reference	9
3.6.1 Detailed Description	10
3.6.2 Member Function Documentation	10
3.6.2.1 feedforward()	10
3.6.2.2 initialize_weights_and_biases()	10
3.6.2.3 load_model_from_file()	11
3.6.2.4 save_model_to_file()	11
3.6.2.5 test_model()	12
3.6.2.6 train()	12
4 File Documentation	13
4.1 NeuralNetworkProjectFoCP/data_preprocessor.h File Reference	13
4.1.1 Detailed Description	13
4.2 data_preprocessor.h	13
4.3 NeuralNetworkProjectFoCP/display.h File Reference	14
4.3.1 Detailed Description	14
4.3.2 Function Documentation	14
4.3.2.1 print_feedforward_output()	14
4.3.2.2 print_header()	15
4.3.2.3 print_vector()	15
4.4 display.h	15
4.5 NeuralNetworkProjectFoCP/file_data_operations.h File Reference	16

4.5.1 Detailed Description	16
4.5.2 Function Documentation	17
4.5.2.1 get_floats_from_line()	17
4.5.2.2 get_strings_from_line()	17
4.5.2.3 load_vector_from_file() [1/3]	17
4.5.2.4 load_vector_from_file() [2/3]	18
4.5.2.5 load_vector_from_file() [3/3]	18
4.5.2.6 parseCSV()	19
4.5.2.7 save_vector_to_file() [1/4]	19
4.5.2.8 save_vector_to_file() [2/4]	19
4.5.2.9 save_vector_to_file() [3/4]	20
4.5.2.10 save_vector_to_file() [4/4]	20
4.6 file_data_operations.h	21
4.7 NeuralNetworkProjectFoCP/math_functions.h File Reference	21
4.7.1 Detailed Description	22
4.7.2 Function Documentation	23
4.7.2.1 cross_entropy()	23
4.7.2.2 cross_entropy_derivative()	23
4.7.2.3 mean_squared_error()	23
4.7.2.4 mean_squared_error_derivative()	24
4.7.2.5 mtanh()	24
4.7.2.6 mtanh_derivative()	25
4.7.2.7 relu()	25
4.7.2.8 relu_derivative()	25
4.7.2.9 sigmoid()	26
4.7.2.10 sigmoid_derivative()	26
4.8 math_functions.h	26
4.9 NeuralNetworkProjectFoCP/neural_network.h File Reference	27
4.9.1 Detailed Description	27
4.10 neural_network.h	27
4.11 resource.h	28
4.12 resource.h	28
4.13 pch.h	28

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

ActivationFunction	Wrapper for an activation function and its derivative. [.	5
ColumnData	Metadata for dataset column	5
Configuration	Holds all runtime settings for the neural network tool	6
DataPreprocessor	Handles the conversion of CSV string data into normalized matrices of floats	6
LossFunction	Wrapper for a loss function and its derivative	9
NeuralNetwork	Implementation of a neural network with multiple functionalities	9

Chapter 2

File Index

2.1 File List

Here is a list of all documented files with brief descriptions:

NeuralNetworkProjectFoCP/ data_preprocessor.h	
Logic for dataset preparation for use in neural network - normalization, and one-hot encoding .	13
NeuralNetworkProjectFoCP/ display.h	
Header file for display functions	14
NeuralNetworkProjectFoCP/ file_data_operations.h	
Declaration of functions for parsing, loading, and saving data to files	16
NeuralNetworkProjectFoCP/ math_functions.h	
Declaration of various mathematical functions and their derivatives	21
NeuralNetworkProjectFoCP/ neural_network.h	
Core Neural Network class definition	27
NeuralNetworkProjectFoCP/ resource.h	28
UnitTest1/ pch.h	28
UnitTest1/ resource.h	28

Chapter 3

Class Documentation

3.1 ActivationFunction Struct Reference

Wrapper for an activation function and its derivative. [.

```
#include <math_functions.h>
```

Public Attributes

- `std::function< std::vector< float >(const std::vector< float > &)>` **func**
- `std::function< std::vector< float >(const std::vector< float > &)>` **derivative**

3.1.1 Detailed Description

Wrapper for an activation function and its derivative. [.

The documentation for this struct was generated from the following file:

- NeuralNetworkProjectFoCP/[math_functions.h](#)

3.2 ColumnData Struct Reference

Metadata for dataset column.

```
#include <data_preprocessor.h>
```

Public Attributes

- `bool` **isNumeric**
- `std::vector< std::string >` **categories**
- `std::unordered_map< std::string, int >` **category_to_index**
- `float` **range** [2]

3.2.1 Detailed Description

Metadata for dataset column.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/data_preprocessor.h](#)

3.3 Configuration Struct Reference

Holds all runtime settings for the neural network tool.

Public Attributes

- `std::string initialization_method` = "Xavier"
- `int hidden_layers` = 1
- `int neurons_nb` = 2
- `std::vector< std::string > activations` = {}
- `std::string train_data_path` = "iris.csv"
- `std::string target_column` = "species"
- `float test_fraction` = 0.2f
- `int epochs` = 140
- `float learning_rate` = 0.2f
- `std::string loss_function` = "BCE"
- `std::string model_save_path` = ""
- `std::string model_load_path` = ""

3.3.1 Detailed Description

Holds all runtime settings for the neural network tool.

This structure is populated by the command-line parser and determines the architecture, data paths, and training hyperparameters.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/main.cpp](#)

3.4 DataPreprocessor Class Reference

Handles the conversion of CSV string data into normalized matrices of floats.

```
#include <data_preprocessor.h>
```

Public Member Functions

- void **clear** ()
Clears all the member variables of the [DataPreprocessor](#).
- void **fit** (const std::vector< std::vector< std::string > > &data)
Fits the preprocessor by determining column types and calculating normalization ranges.
- std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > > **transform_and_extract** (const std::vector< std::vector< std::string > > &data, const std::string &column_to_extract, bool is_there_header=true)
Transforms the dataset using One-Hot encoding and Min-Max normalization.
- void **interpret_extracted_column** (const std::vector< float > &transformed_data) const
Decodes processed float data back into human-readable format.
- void **print_state** () const
Prints the current state of the [DataPreprocessor](#).

3.4.1 Detailed Description

Handles the conversion of CSV string data into normalized matrices of floats.

3.4.2 Member Function Documentation

3.4.2.1 fit()

```
void DataPreprocessor::fit (
    const std::vector< std::vector< std::string > > & data)
```

Fits the preprocessor by determining column types and calculating normalization ranges.

Iterates through the dataset to identify which column is numeric and which is categorical. For numeric columns, it finds min/max for normalization. For categorical, it maps unique strings to indices.

Parameters

<i>data</i>	Dataset (2D string vector). The first row must be the header.
-------------	---

Note

This function populates internal maps and clears any previous state.

3.4.2.2 interpret_extracted_column()

```
void DataPreprocessor::interpret_extracted_column (
    const std::vector< float > & transformed_data) const
```

Decodes processed float data back into human-readable format.

Reverses normalization for numeric columns and picks the highest probability category for one-hot encoded categorical columns.

Parameters

<i>transformed_data</i>	A 1D vector representing a single sample of the extracted column.
-------------------------	---

Precondition

[transform_and_extract\(\)](#) must have been called at least once to set the extracted column name.

3.4.2.3 transform_and_extract()

```
std::pair< std::vector< std::vector< float > >, std::vector< std::vector< float > > >
DataPreprocessor::transform_and_extract (
    const std::vector< std::vector< std::string > > & data,
    const std::string & column_to_extract,
    bool is_there_header = true)
```

Transforms the dataset using One-Hot encoding and Min-Max normalization.

Separates the dataset into a feature set and a specific target column.

Parameters

<i>data</i>	The raw string data to transform.
<i>column_to_extract</i>	The name of the column to be used as the target.
<i>is_there_header</i>	Flag indicating if the input dataset includes a header row.

Returns

A pair of 2D float vectors:

- **first**: Transformed feature matrix (all columns except the extracted one).
- **second**: Transformed target matrix (the extracted column).

Precondition

The preprocessor must have been successfully fitted using [fit\(\)](#).

The documentation for this class was generated from the following files:

- [NeuralNetworkProjectFoCP/data_preprocessor.h](#)
- [NeuralNetworkProjectFoCP/data_preprocessor.cpp](#)

3.5 LossFunction Struct Reference

Wrapper for a loss function and its derivative.

```
#include <math_functions.h>
```

Public Attributes

- `std::function< std::vector< float > (const std::vector< float > &, const std::vector< float > &)>` **func**
- `std::function< std::vector< float > (const std::vector< float > &, const std::vector< float > &)>` **derivative**

3.5.1 Detailed Description

Wrapper for a loss function and its derivative.

The documentation for this struct was generated from the following file:

- [NeuralNetworkProjectFoCP/math_functions.h](#)

3.6 NeuralNetwork Class Reference

Implementation of a neural network with multiple functionalities.

```
#include <neural_network.h>
```

Public Member Functions

- void [initialize_weights_and_biases](#) (int input_size, int hidden_layers_number, int neurons_per_hidden_layer, int output_size, std::string initialization_method="Xavier", std::vector< std::string > activation_functions={}, int seed=std::random_device>())
Initialize weights and biases for the neural network.
- void **visualize_model** () const
Visualize the neural network model structure (weights and biases).
- std::vector< std::vector< float > > [feedforward](#) (const std::vector< std::vector< float > > &input) const
Performs a forward pass through the network for multiple input samples.
- void [train](#) (const std::vector< std::vector< float > > &inputs, const std::vector< std::vector< float > > &targets, const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, int epochs, float learning_rate, const std::string &loss_function_name)
Train the neural network using the provided training data.
- float [test_model](#) (const std::vector< std::vector< float > > &test_inputs, const std::vector< std::vector< float > > &test_targets, const std::string &loss_function_name) const
Calculates the network loss on a test dataset.
- bool [save_model_to_file](#) (const std::string &filename) const
Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.
- bool [load_model_from_file](#) (const std::string &filename)
Loads the model state (weights, biases, activations) from files.
- const std::vector< std::vector< std::vector< float > > > & **get_weights** () const
- const std::vector< std::vector< float > > & **get_biases** () const
- const std::vector< std::string > & **get_activations** () const

3.6.1 Detailed Description

Implementation of a neural network with multiple functionalities.

3.6.2 Member Function Documentation

3.6.2.1 feedforward()

```
std::vector< std::vector< float > > NeuralNetwork::feedforward (
    const std::vector< std::vector< float > > & input) const
```

Performs a forward pass through the network for multiple input samples.

Parameters

<i>input</i>	A 2D vector where each row is an input sample.
--------------	--

Returns

A 2D vector of predictions. Returns an empty 2D vector if the model is invalid.

Note

Ensure the input sample size matches the network's input layer size.

3.6.2.2 initialize_weights_and_biases()

```
void NeuralNetwork::initialize_weights_and_biases (
    int input_size,
    int hidden_layers_number,
    int neurons_per_hidden_layer,
    int output_size,
    std::string initialization_method = "Xavier",
    std::vector< std::string > activation_functions = {},
    int seed = std::random_device()())
```

Initialize weights and biases for the neural network.

Supports Xavier and He initialization methods. Biases are initialized to zero.

Parameters

<i>input_size</i>	The size of the input layer.
<i>hidden_layers_number</i>	The number of hidden layers.
<i>neurons_per_hidden_layer</i>	The number of neurons in each hidden layer.
<i>output_size</i>	The size of the output layer.

<i>initialization_method</i>	The method used ("Xavier" or "He"). Defaults is Xavier.
<i>activation_functions</i>	Activation functions for each layer (count should be hidden_layers_number + 1).
<i>seed</i>	The random seed for reproducibility.

3.6.2.3 load_model_from_file()

```
bool NeuralNetwork::load_model_from_file (  
    const std::string & filename)
```

Loads the model state (weights, biases, activations) from files.

Parameters

<i>filename</i>	The base name of the files (appends _weights.txt etc automatically).
-----------------	--

Note

This function calls `validate_model_structure()` after loading.

Returns

True if successful, false otherwise.

3.6.2.4 save_model_to_file()

```
bool NeuralNetwork::save_model_to_file (  
    const std::string & filename) const
```

Saves the model state (weights, biases, activations) to 3 files: {filename}_weights.txt, {filename}_biases.txt, {filename}_activations.txt.

Parameters

<i>filename</i>	The base name of the files (appends _weights.txt etc. automatically).
-----------------	---

Returns

True if successful, false otherwise.

3.6.2.5 test_model()

```
float NeuralNetwork::test_model (
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    const std::string & loss_function_name) const
```

Calculates the network loss on a test dataset.

Parameters

<i>test_inputs</i>	Input samples for testing.
<i>test_targets</i>	Expected output labels.
<i>loss_function_name</i>	The name of the loss function ("MSE" or "BCE").

Returns

The average loss across all samples. Returns -1.0f if the model is invalid or sizes mismatch.

3.6.2.6 train()

```
void NeuralNetwork::train (
    const std::vector< std::vector< float > > & inputs,
    const std::vector< std::vector< float > > & targets,
    const std::vector< std::vector< float > > & test_inputs,
    const std::vector< std::vector< float > > & test_targets,
    int epochs,
    float learning_rate,
    const std::string & loss_function_name)
```

Train the neural network using the provided training data.

Parameters

<i>inputs</i>	The input data for training - each row represents a training sample. It is expected to be shuffled.
<i>targets</i>	The target output data for training - each row represents the target output for the corresponding training sample.
<i>test_inputs</i>	The input data for testing during training - each row represents a different test sample.
<i>test_targets</i>	The target output data for testing during training - each row represents the target output for the corresponding test sample.
<i>epochs</i>	The number of epochs to train for.
<i>learning_rate</i>	The learning rate for weight updates.
<i>loss_function_name</i>	The name of the loss function to use - either "MSE" or "BCE".

The documentation for this class was generated from the following files:

- NeuralNetworkProjectFoCP/[neural_network.h](#)
- NeuralNetworkProjectFoCP/neural_network.cpp

Chapter 4

File Documentation

4.1 NeuralNetworkProjectFoCP/data_preprocessor.h File Reference

Logic for dataset preparation for use in neural network - normalization, and one-hot encoding.

```
#include <vector>
#include <string>
#include <unordered_map>
```

Classes

- struct [ColumnData](#)
Metadata for dataset column.
- class [DataPreprocessor](#)
Handles the conversion of CSV string data into normalized matrices of floats.

4.1.1 Detailed Description

Logic for dataset preparation for use in neural network - normalization, and one-hot encoding.

4.2 data_preprocessor.h

[Go to the documentation of this file.](#)

```
00001 #ifndef DATAPREPROCESSOR_H
00002 #define DATAPREPROCESSOR_H
00006 #include <vector>
00007 #include <string>
00008 #include <unordered_map>
00009
00014 struct ColumnData {
00015     bool isNumeric;
00016     std::vector<std::string> categories; // All unique categories for categorical columns
00017     std::unordered_map<std::string, int> category_to_index;
00018     float range[2]; // Min and Max for numeric columns
00019 };
00024 class DataPreprocessor {
00025 private:
00026     std::vector<ColumnData> columns; // Column index -> column_data
```

```

00027     std::unordered_map<std::string, int> column_order;
00028     std::string extracted_column_name;
00029     bool is_fitted = false;
00030
00031 public:
00032     void clear();
00043     void fit(const std::vector<std::vector<std::string>& data);
00055     std::pair<std::vector<std::vector<float>>, std::vector<std::vector<float>>>
        transform_and_extract(const std::vector<std::vector<std::string>& data, const std::string&
        column_to_extract, bool is_there_header = true);
00063     void interpret_extracted_column(const std::vector<float>& transformed_data) const;
00067     void print_state() const;
00068 };
00069
00070 #endif // DATAPREPROCESSOR_H

```

4.3 NeuralNetworkProjectFoCP/display.h File Reference

Header file for display functions.

```
#include <string>
```

Functions

- void `print_header` (const std::string &text)
Prints a stylized header to the console.
- void `print_vector` (const std::vector< float > &vec)
Prints a 1D vector of floats in a formatted horizontal array.
- void `print_feedforward_output` (const std::vector< std::vector< float > > &feedforward_input, const std::vector< std::vector< float > > &target_output, const std::vector< std::vector< float > > &feedforward_output, const size_t sample_count=5)
Displays a comparison between inputs, targets, and actual model predictions.

4.3.1 Detailed Description

Header file for display functions.

4.3.2 Function Documentation

4.3.2.1 `print_feedforward_output()`

```

void print_feedforward_output (
    const std::vector< std::vector< float > > & feedforward_input,
    const std::vector< std::vector< float > > & target_output,
    const std::vector< std::vector< float > > & feedforward_output,
    const size_t sample_count = 5)

```

Displays a comparison between inputs, targets, and actual model predictions.

Useful for visually inspecting model performance on a small sample of data.

Parameters

<i>feedforward_input</i>	The original feature data.
<i>target_output</i>	The true, target labels.
<i>feedforward_output</i>	The predictions of the NeuralNetwork .
<i>sample_count</i>	The maximum number of samples to print (5 is default).

4.3.2.2 print_header()

```
void print_header (
    const std::string & text)
```

Prints a stylized header to the console.

The header is surrounded by "=" characters from upper and lower side.

Parameters

<i>text</i>	The string to be displayed within the header.
-------------	---

4.3.2.3 print_vector()

```
void print_vector (
    const std::vector< float > & vec)
```

Prints a 1D vector of floats in a formatted horizontal array.

Output format is [a b c ...] with each value fixed to 2 decimal places and a width of 6 characters for alignment.

Parameters

<i>vec</i>	The vector of floats to display.
------------	----------------------------------

4.4 display.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00002 #ifndef DISPLAY_H
00003 #define DISPLAY_H
00004
00005 #include <string>
00009
00015 void print_header(const std::string& text);
00022 void print_vector(const std::vector<float>& vec);
00023
00032 void print_feedforward_output(const std::vector<std::vector<float>& feedforward_input,
00033                               const std::vector<std::vector<float>& target_output,
00034                               const std::vector<std::vector<float>& feedforward_output,
00035                               const size_t sample_count = 5);
00036 #endif
```

4.5 NeuralNetworkProjectFoCP/file_data_operations.h File Reference

Declaration of functions for parsing, loading, and saving data to files.

```
#include <vector>
#include <string>
#include <fstream>
```

Functions

- `std::vector< std::string > get\_strings\_from\_line (const std::string &line)`
Convert a line of comma-separated values into a vector of strings.
- `std::vector< float > get\_floats\_from\_line (const std::string &line)`
Get a vector of floats from a comma-separated line.
- `std::vector< std::vector< std::string > > parseCSV (const std::string &filename)`
Loading CSV file and parsing data into a string matrix.
- `bool save\_vector\_to\_file (const std::string &filename, const std::vector< float > &vec, char delimiter=',')`
Save a 1D vector of floats to a file.
- `bool save\_vector\_to\_file (const std::string &filename, const std::vector< std::string > &vec, char delimiter=',')`
Save a 1D vector of strings to a file.
- `bool save\_vector\_to\_file (const std::string &filename, const std::vector< std::vector< float > > &vec, char delimiter=',')`
Save a 2D vector of floats to a file.
- `bool save\_vector\_to\_file (const std::string &filename, const std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')`
Save a 3D vector of floats to a file.
- `bool load\_vector\_from\_file (const std::string &filename, std::vector< std::string > &vec, char delimiter=',')`
Load a 1D vector of strings from a single line file.
- `bool load\_vector\_from\_file (const std::string &filename, std::vector< std::vector< float > > &vec, char delimiter=',')`
Load a 2D vector of floats from a file.
- `bool load\_vector\_from\_file (const std::string &filename, std::vector< std::vector< std::vector< float > > > &vec, char delimiter=',')`
Load a 3D vector from a file.

4.5.1 Detailed Description

Declaration of functions for parsing, loading, and saving data to files.

4.5.2 Function Documentation

4.5.2.1 get_floats_from_line()

```
std::vector< float > get_floats_from_line (
    const std::string & line)
```

Get a vector of floats from a comma-separated line.

Parameters

<i>line</i>	String containing comma-separated float values.
-------------	---

Returns

1D vector of floats.

4.5.2.2 get_strings_from_line()

```
std::vector< std::string > get_strings_from_line (
    const std::string & line)
```

Convert a line of comma-separated values into a vector of strings.

Parameters

<i>line</i>	String line containing comma-separated values.
-------------	--

Returns

1D vector of strings.

4.5.2.3 load_vector_from_file() [1/3]

```
bool load_vector_from_file (
    const std::string & filename,
    std::vector< std::string > & vec,
    char delimiter = ',')
```

Load a 1D vector of strings from a single line file.

Parameters

	<i>filename</i>	The name of the file to read (".txt" must be included).
out	<i>vec</i>	The string vector to be populated (it is cleared before).
	<i>delimiter</i>	The character used to split the values.

Returns

True if the operation was successful, false otherwise.

4.5.2.4 load_vector_from_file() [2/3]

```
bool load_vector_from_file (
    const std::string & filename,
    std::vector< std::vector< float > > & vec,
    char delimiter = ',')
```

Load a 2D vector of floats from a file.

Each non-empty line in the file is treated as a row in the 2D vector.

Parameters

	<i>filename</i>	The name of the file to read.
out	<i>vec</i>	The 2D vector to be populated.
	<i>delimiter</i>	The character used to separate values.

Returns

True if the operation was successful, false otherwise. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.5 load_vector_from_file() [3/3]

```
bool load_vector_from_file (
    const std::string & filename,
    std::vector< std::vector< std::vector< float > > > & vec,
    char delimiter = ',')
```

Load a 3D vector from a file.

Reconstructs the 3D structure by using empty lines as separators between different 2D vectors.

Parameters

	<i>filename</i>	The name of the file to read.
out	<i>vec</i>	The 3D vector to be populated.
	<i>delimiter</i>	The character used to separate values.

Returns

True if the operation was successful, false otherwise. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.6 parseCSV()

```
std::vector< std::vector< std::string > > parseCSV (
    const std::string & filename)
```

Loading CSV file and parsing data into a string matrix.

It reads all lines - including the header.

Parameters

<i>filename</i>	The name of the CSV file to parse (".csv" must be included).
-----------------	--

Returns

2D vector (rows and columns) of strings.

4.5.2.7 save_vector_to_file() [1/4]

```
bool save_vector_to_file (
    const std::string & filename,
    const std::vector< float > & vec,
    char delimiter = ',')
```

Save a 1D vector of floats to a file.

Parameters

<i>filename</i>	Name of the file to create or overwrite.
<i>vec</i>	The 1D vector containing float data.
<i>delimiter</i>	The character used to separate values (default is comma).

Returns

True if the operation was successful, false otherwise.

4.5.2.8 save_vector_to_file() [2/4]

```
bool save_vector_to_file (
    const std::string & filename,
    const std::vector< std::string > & vec,
    char delimiter = ',')
```

Save a 1D vector of strings to a file.

Parameters

<i>filename</i>	Name of the file to create or overwrite.
<i>vec</i>	The 1D vector containing string data.
<i>delimiter</i>	The character used to separate values (default is comma).

Returns

True if the operation was successful, false otherwise. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.9 `save_vector_to_file()` [3/4]

```
bool save_vector_to_file (
    const std::string & filename,
    const std::vector< std::vector< float > > & vec,
    char delimiter = ',')
```

Save a 2D vector of floats to a file.

Each inner vector is written as a single line, with elements separated by the delimiter.

Parameters

<i>filename</i>	Name of the file to create or overwrite.
<i>vec</i>	The 2D vector of floats.
<i>delimiter</i>	The character used to separate values.

Returns

True if the operation was successful, false otherwise. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.5.2.10 `save_vector_to_file()` [4/4]

```
bool save_vector_to_file (
    const std::string & filename,
    const std::vector< std::vector< std::vector< float > > > & vec,
    char delimiter = ',')
```

Save a 3D vector of floats to a file.

2D vectors are separated by a blank line. Within each 2D vector, rows are separated by newlines and elements by the delimiter.

Parameters

<i>filename</i>	Name of the file to create or overwrite.
-----------------	--

<i>vec</i>	The 3D vector to save.
<i>delimiter</i>	The character used to separate values.

Returns

True if the operation was successful, false otherwise. This is an overloaded member function, provided for convenience. It differs from the above function only in what argument(s) it accepts.

4.6 file_data_operations.h

[Go to the documentation of this file.](#)

```

00001 #pragma once
00002 #ifndef FILE_DATA_OPERATIONS_H
00003 #define FILE_DATA_OPERATIONS_H
00004
00008
00009 #include <vector>
00010 #include <string>
00011 #include <fstream>
00012
00013 //Parsing functions
00019 std::vector<std::string> get_strings_from_line(const std::string& line);
00020
00026 std::vector<float> get_floats_from_line(const std::string& line);
00027
00028 //Loading/saving to file functions
00035 std::vector<std::vector<std::string>> parseCSV(const std::string& filename);
00036
00037 // Overload for 1D vectors
00045 bool save_vector_to_file(const std::string& filename,
00046     const std::vector<float>& vec,
00047     char delimiter = ',');
00048
00057 bool save_vector_to_file(const std::string& filename, const std::vector<std::string>& vec, char
    delimiter = ',');
00058
00059 // Overload for 2D vectors
00069 bool save_vector_to_file(const std::string& filename,
00070     const std::vector<std::vector<float>>& vec,
00071     char delimiter = ',');
00072
00073 // Overload for 3D vectors
00084 bool save_vector_to_file(const std::string& filename, const
    std::vector<std::vector<std::vector<float>>>& vec, char delimiter = ',');
00085
00086 // Overloads for loading from file
00094 bool load_vector_from_file(const std::string& filename, std::vector<std::string>& vec, char delimiter
    = ',');
00095
00096 // 2D vector overload
00106 bool load_vector_from_file(const std::string& filename, std::vector<std::vector<float>>& vec, char
    delimiter = ',');
00107
00108 // 3D vector overload
00118 bool load_vector_from_file(const std::string& filename, std::vector<std::vector<std::vector<float>>>&
    vec, char delimiter = ',');
00119
00120 #endif

```

4.7 NeuralNetworkProjectFoCP/math_functions.h File Reference

Declaration of various mathematical functions and their derivatives.

```

#include <vector>
#include <string>
#include <unordered_map>
#include <functional>

```

Classes

- struct [ActivationFunction](#)
Wrapper for an activation function and its derivative.
- struct [LossFunction](#)
Wrapper for a loss function and its derivative.

Functions

- `std::vector< float > relu (const std::vector< float > &x)`
Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.
- `std::vector< float > relu_derivative (const std::vector< float > &x)`
Derivative of the ReLU function: $f'(x) = 1$ if $x > 0$ else 0 .
- `std::vector< float > sigmoid (const std::vector< float > &x)`
Applies the sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.
- `std::vector< float > sigmoid_derivative (const std::vector< float > &x)`
Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.
- `std::vector< float > mtanh (const std::vector< float > &x)`
Applies the hyperbolic tangent (Tanh) activation: $f(x) = \tanh(x)$.
- `std::vector< float > mtanh_derivative (const std::vector< float > &x)`
Applies the derivative of the Tanh activation on vector - $1 - \tanh^2(x)$.
- `std::vector< float > mean_squared_error (const std::vector< float > &predicted, const std::vector< float > &actual)`
Computes the Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.
- `std::vector< float > mean_squared_error_derivative (const std::vector< float > &predicted, const std::vector< float > &actual)`
Computes the derivative of the MSE loss with respect to the prediction.
- `std::vector< float > cross_entropy (const std::vector< float > &predicted, const std::vector< float > &actual)`
Binary Cross Entropy Loss: $L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$.
- `std::vector< float > cross_entropy_derivative (const std::vector< float > &predicted, const std::vector< float > &actual)`
Derivative of Binary Cross Entropy Loss with respect to predictions.

Variables

- `const std::unordered_map< std::string, ActivationFunction > activation_map`
Maps string names (e.g., "relu") to [ActivationFunction](#) objects.
- `const std::unordered_map< std::string, LossFunction > loss_function_map`
Maps string names (e.g., "MSE") to [LossFunction](#) objects.

4.7.1 Detailed Description

Declaration of various mathematical functions and their derivatives.

4.7.2 Function Documentation

4.7.2.1 cross_entropy()

```
std::vector< float > cross_entropy (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Binary Cross Entropy Loss: $L = -[y \cdot \log(p) + (1-y) \cdot \log(1-p)]$.

Parameters

<i>predicted</i>	The vector of predicted probabilities (must be in range 0-1).
<i>actual</i>	The true target labels (0 or 1).

Returns

A vector of cross-entropy losses.

4.7.2.2 cross_entropy_derivative()

```
std::vector< float > cross_entropy_derivative (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Derivative of Binary Cross Entropy Loss with respect to predictions.

Parameters

<i>predicted</i>	The vector of predicted probabilities (must be in range 0-1).
<i>actual</i>	The true target labels (0 or 1).

Returns

Gradient vector.

4.7.2.3 mean_squared_error()

```
std::vector< float > mean_squared_error (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Computes the Squared Error loss: $L = \frac{1}{2}(pred - actual)^2$.

Parameters

<i>predicted</i>	The output predicted by the network.
<i>actual</i>	The true target values.

Returns

A vector of squared differences.

4.7.2.4 mean_squared_error_derivative()

```
std::vector< float > mean_squared_error_derivative (
    const std::vector< float > & predicted,
    const std::vector< float > & actual)
```

Computes the derivative of the MSE loss with respect to the prediction.

$$\frac{\partial L}{\partial pred} = pred - actual$$

Parameters

<i>predicted</i>	The predicted output vector.
<i>actual</i>	The actual target vector.

Returns

Gradient vector.

4.7.2.5 mtanh()

```
std::vector< float > mtanh (
    const std::vector< float > & x)
```

Applies the hyperbolic tangent (Tanh) activation: $f(x) = \tanh(x)$.

Parameters

<i>x</i>	Input vector.
----------	---------------

Returns

Vector with tanh values in range (-1, 1).

4.7.2.6 mtanh_derivative()

```
std::vector< float > mtanh_derivative (
    const std::vector< float > & x)
```

Applies the derivative of the Tanh activation on vector - $1 - \tanh^2(x)$.

Parameters

x	Input vector.
-----	---------------

Returns

Gradient vector.

4.7.2.7 relu()

```
std::vector< float > relu (
    const std::vector< float > & x)
```

Applies the Rectified Linear Unit (ReLU) activation: $f(x) = \max(0, x)$.

Parameters

x	Input vector.
-----	---------------

Returns

Vector with ReLU applied.

4.7.2.8 relu_derivative()

```
std::vector< float > relu_derivative (
    const std::vector< float > & x)
```

Derivative of the ReLU function: $f'(x) = 1$ if $x > 0$ else 0.

Parameters

x	Input vector.
-----	---------------

Returns

Gradient vector.

4.7.2.9 sigmoid()

```
std::vector< float > sigmoid (
    const std::vector< float > & x)
```

Applies the sigmoid activation: $f(x) = \frac{1}{1+e^{-x}}$.

Parameters

x	Input vector.
---	---------------

Returns

Vector with sigmoid values in range (0, 1).

4.7.2.10 sigmoid_derivative()

```
std::vector< float > sigmoid_derivative (
    const std::vector< float > & x)
```

Derivative of the Sigmoid function: $f'(x) = f(x) \cdot (1 - f(x))$.

Parameters

x	Input vector.
---	---------------

Returns

Gradient vector.

4.8 math_functions.h

[Go to the documentation of this file.](#)

```
00001 #pragma once
00005
00006 #include <vector>
00007 #include <string>
00008 #include <unordered_map>
00009 #include <functional>
00010
00016 std::vector<float> relu(const std::vector<float>& x);
00022 std::vector<float> relu_derivative(const std::vector<float>& x);
00023
00029 std::vector<float> sigmoid(const std::vector<float>& x);
00035 std::vector<float> sigmoid_derivative(const std::vector<float>& x);
00036
00042 std::vector<float> mtanh(const std::vector<float>& x);
00048 std::vector<float> mtanh_derivative(const std::vector<float>& x);
00049
00050
00057 std::vector<float> mean_squared_error(const std::vector<float>& predicted, const std::vector<float>&
actual);
00065 std::vector<float> mean_squared_error_derivative(const std::vector<float>& predicted, const
std::vector<float>& actual);
00066
00073 std::vector<float> cross_entropy(const std::vector<float>& predicted, const std::vector<float>&
actual);
```

```

00080 std::vector<float> cross_entropy_derivative(const std::vector<float>& predicted, const
      std::vector<float>& actual);
00085 struct ActivationFunction {
00086     std::function<std::vector<float>(const std::vector<float>&)> func;
00087     std::function<std::vector<float>(const std::vector<float>&)> derivative;
00088 };
00094 struct LossFunction {
00095     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)> func;
00096     std::function<std::vector<float>(const std::vector<float>&, const std::vector<float>&)>
      derivative;
00097 };
00099 extern const std::unordered_map<std::string, ActivationFunction> activation_map;
00101 extern const std::unordered_map<std::string, LossFunction> loss_function_map;

```

4.9 NeuralNetworkProjectFoCP/neural_network.h File Reference

Core Neural Network class definition.

```

#include <vector>
#include <string>
#include <random>

```

Classes

- class [NeuralNetwork](#)
Implementation of a neural network with multiple functionalities.

4.9.1 Detailed Description

Core Neural Network class definition.

4.10 neural_network.h

[Go to the documentation of this file.](#)

```

00001 #ifndef NEURAL_NETWORK_H
00002 #define NEURAL_NETWORK_H
00006 #include <vector>
00007 #include <string>
00008 #include <random>
00009
00014 class NeuralNetwork {
00015 public:
00016     NeuralNetwork();
00028     void initialize_weights_and_biases(int input_size, int hidden_layers_number, int
      neurons_per_hidden_layer, int output_size, std::string initialization_method = "Xavier",
00029     std::vector<std::string> activation_functions = {},
00030     int seed = std::random_device()());
00034     void visualize_model() const;
00041     std::vector<std::vector<float>> feedforward(const std::vector<std::vector<float>>& input) const;
00052     void train(const std::vector<std::vector<float>>& inputs,
00053     const std::vector<std::vector<float>>& targets,
00054     const std::vector<std::vector<float>>& test_inputs,
00055     const std::vector<std::vector<float>>& test_targets,
00056     int epochs,
00057     float learning_rate,
00058     const std::string& loss_function_name);
00066     float test_model(const std::vector<std::vector<float>>& test_inputs,
00067     const std::vector<std::vector<float>>& test_targets,
00068     const std::string& loss_function_name) const;
00074     bool save_model_to_file(const std::string& filename) const;
00081     bool load_model_from_file(const std::string& filename);
00082

```

```

00083 // Getters (mainly for testing)
00084 const std::vector<std::vector<std::vector<float>>>& get_weights() const;
00085 const std::vector<std::vector<float>>& get_biases() const;
00086 const std::vector<std::string>& get_activations() const;
00087
00088 private:
00089     std::vector<std::vector<std::vector<float>>> weights;
00090     std::vector<std::vector<float>> biases;
00091     std::vector<std::string> activations;
00092
00093     bool is_model_valid = false;
00094     void validate_model_structure();
00095
00096 };
00097 #endif

```

4.11 resource.h

```

00001 //{NO_DEPENDENCIES}
00002 // used by: NeuralNetworkProjectFoCP.rc
00003 //
00004 #define IDR_MENU1 101
00005 #define ID_HEJ_HWEH 40001
00006
00007 // Next default values for new objects
00008 //
00009 #ifdef APSTUDIO_INVOKED
00010 #ifndef APSTUDIO_READONLY_SYMBOLS
00011 #define _APS_NEXT_RESOURCE_VALUE 102
00012 #define _APS_NEXT_COMMAND_VALUE 40002
00013 #define _APS_NEXT_CONTROL_VALUE 1001
00014 #define _APS_NEXT_SYMED_VALUE 101
00015 #endif
00016 #endif

```

4.12 resource.h

```

00001 //{NO_DEPENDENCIES}
00002 // Microsoft Visual C++ generated include file.
00003 // Used by UnitTest1.rc
00004
00005 // Następnne wartości domyślne dla nowych obiektów
00006 //
00007 #ifdef APSTUDIO_INVOKED
00008 #ifndef APSTUDIO_READONLY_SYMBOLS
00009 #define _APS_NEXT_RESOURCE_VALUE 101
00010 #define _APS_NEXT_COMMAND_VALUE 40001
00011 #define _APS_NEXT_CONTROL_VALUE 1001
00012 #define _APS_NEXT_SYMED_VALUE 101
00013 #endif
00014 #endif

```

4.13 pch.h

```

00001 // pch.h: wstępnie skompilowany plik nagłówka.
00002 // Wymienione poniżej pliki są kompilowane tylko raz, co poprawia wydajność kompilacji dla przyszłych
    kompilacji.
00003 // Ma to także wpływ na wydajność funkcji IntelliSense, w tym uzupełnianie kodu i wiele funkcji
    przeglądania kodu.
00004 // Jednak WSZYSTKIE wymienione tutaj pliki będą ponownie kompilowane, jeśli którykolwiek z nich
    zostanie zaktualizowany między kompilacjami.
00005 // Nie dodawaj tutaj plików, które będziesz często aktualizować (obniża to korzystny wpływ na
    wydajność).
00006
00007 #ifndef PCH_H
00008 #define PCH_H
00009
00010 // w tym miejscu dodaj nagłówki, które mają być wstępnie kompilowane
00011
00012 #endif //PCH_H

```